

Module 7 Lab: Session 0: The M6→M7 Handoff (Split into Clean Architecture)

Module	M7: Advanced Web API Development
Session	0 of 4 (pre-session, not graded)
What you do	Split the single M6 TmsApi project into the four-project clean-architecture shape the rest of M7 assumes

Why this exists

In M6 you shipped a working TMS API. It is one ASP.NET project: `TmsApi.csproj`: with `Entities/`, `Data/`, `Services/`, `DTOs/`, and `Controllers/` folders all in the same assembly. The Angular team in M8 and every integration partner downstream depend on that API being *correct* and *production-shaped*.

“Production-shaped” in M7 means **clean architecture**: the rule the rest of the labs assume is that **dependencies point inward** (the outer Api layer can see the inner layers; the inner layers cannot see the outer ones). M7’s Session 1 exercise 2 opens with `cd TmsApi.Api && dotnet run` and then immediately writes `TmsApi.Application/Common/Result.cs`, `TmsApi.Infrastructure/Caching/...`, `TmsApi.Api/Controllers/V1/...`. None of those paths exist in the M6 monolith. Session 0 is that turns the M6 monolith into the M7 four-project solution.

If you skip this and try to follow Session 1, the first `dotnet add TmsApi.Api package ...` command will fail because `TmsApi.Api` does not exist. If you try to “wing it” by hand, you will spend the entire Session 1 debugging namespace mismatches instead of learning versioning and CQRS. Two hours now saves a full day later.

Windows Users: Shell Recommendation: We recommend using **Git Bash** (installed by default with Git) for all terminal commands in this guide. This ensures that Unix utility commands (`mv`, `mkdir`, `grep`, `rmdir`) work identically on your machine without requiring conversion to PowerShell equivalents.

Words you need before you start

You will see these terms in every step. Read this once and refer back when something doesn't ring a bell.

- **.csproj**: an XML file at the root of every C# project. It declares the SDK (what kind of project it is), the target framework, and the package/project references. You will be opening and editing .csproj files in Step 1.
- **SDK**: the "kit" that knows how to build a project. A `classlib` SDK builds a class library (no entry point, no web). The web SDK (full name `Microsoft.NET.Sdk.Web`) builds an ASP.NET application.
- **Project reference** vs **package reference**: both are listed inside a .csproj, but they are different. A *project reference* points to another .csproj in the same workspace (`<ProjectReference Include="../TmsApi.Domain/TmsApi.Domain.csproj" />`): it lets one project use another project's classes. A *package reference* points to NuGet packages. We use both.
- **Startup project**: the project `dotnet run` (and `dotnet ef`) actually launches. It is the project that has `Program.cs` and the web SDK. In the M7 shape, the startup project is **always** `TmsApi.Api`.
- **`dotnet new classlib -n X -f net10.0`**: create a new class library project named `X` targeting .NET 10.0.
- **`dotnet add Y reference Z`**: add a *project reference* from project `Y` to project `Z`.
- **`dotnet sln add Y`**: register project `Y` inside the solution file (`TmsApi.sln`).

Before You Begin: confirm the M6 baseline

You should already have completed M6 with a working TMS API exposing GET `/api/courses`, GET `/api/courses/{id}`, POST `/api/courses/{courseId}/enrollments`, `DataSeeder` running on dev startup, and `Scalar` at the path your M6 project mapped (typically `https://localhost:5001/scalar/v1`, or `/scalar`).

From the project root:

```
dotnet build
dotnet run --project TmsApi.csproj
```

You should see the API start on `https://localhost:5001` (or whatever HTTPS URL is in your `launchSettings.json`). **Stop here and fix any M6 build errors**

before continuing. Session 0 stacks on a working M6 baseline: a broken build wastes the whole refactor.

Commit the baseline before continuing:

```
git add -A
git commit -m "M6: green baseline before M7 clean-architecture split"
```

The shape we are splitting into

Read this once before you start moving files. The Onion / Clean Architecture rule is **dependencies point inward**. The outer layer (Api) knows about every inner layer. The inner layers know nothing about the outer ones. The compiler enforces the rule for free once the projects are wired correctly: `TmsApi.Domain.csproj` will literally refuse to reference `TmsApi.Infrastructure`, because the project reference does not exist.

Project	SDK	References	Holds
<code>TmsApi.Domain</code>	classlib	(none)	Pure C# entities, value objects, domain events. No NuGet. No EF. No ASP.NET.
<code>TmsApi.Application</code>	classlib	<code>TmsApi.Domain</code>	DTOs, service interfaces, MediatR commands/queries/handlers, <code>Result<T,E></code> , pipeline behaviors.
<code>TmsApi.Infrastructure</code>	classlib	<code>TmsApi.Application</code> , <code>TmsApi.Domain</code>	EF Core <code>TmsDbContext</code> + <code>IEntityTypeConfiguration<></code> s, repository implementations, external HTTP clients, <code>HybridCache</code> adapter.
<code>TmsApi.Api</code>	web	<code>TmsApi.Application</code> , <code>TmsApi.Infrastructure</code> , <code>TmsApi.Domain</code>	Controllers, middleware, <code>Program.cs</code> , <code>appsettings.json</code> , <code>Program.cs</code> -level DI registration.

The M6 monolith's `TmsApi.csproj` keeps the web project (`TmsApi.Api`) but **must** be renamed so the new `TmsApi.Api.csproj` can take over. Step 1 renames the existing `.csproj` to `TmsApi.Api.csproj` and switches it to the web SDK.

Why four projects, not three? You will see some M7 handouts put the domain entities in `TmsApi.Application` and collapse to three projects. Either shape is valid; the four-project version is the M7 standard because (a) the Domain layer never needs to take a MediatR or FluentValidation dependency, (b) when you eventually split the worker out into its own process for Exercise 5, the worker can reference `TmsApi.Application` without dragging in the entire web project, and (c) the four-project shape mirrors Section 8 of the M7 essentials verbatim, which makes the M7 capstone grading consistent with the diagram.

Step 1: Rename the M6 project to `TmsApi.Api` and create the three new projects

You will perform this step from the **project root directory** (where your existing `TmsApi.csproj` and `TmsApi.sln` reside).

1. Create the `TmsApi.Api` directory and move/rename the project file

Create the directory for your API project, rename the existing monolith's `.csproj` file, and move it inside the new folder. Run these commands in your Git Bash terminal:

```
mkdir TmsApi.Api
mv TmsApi.csproj TmsApi.Api/TmsApi.Api.csproj
```

2. Create the three new class libraries

Run the following commands to create the other Clean Architecture layers:

```
dotnet new classlib -n TmsApi.Domain -f net10.0
dotnet new classlib -n TmsApi.Application -f net10.0
dotnet new classlib -n TmsApi.Infrastructure -f net10.0
```

3. Register all four projects in your solution file

Add the newly created class libraries and the renamed API project to the solution:

```
dotnet sln TmsApi.sln add TmsApi.Domain/TmsApi.Domain.csproj
dotnet sln TmsApi.sln add TmsApi.Application/TmsApi.Application.csproj
dotnet sln TmsApi.sln add TmsApi.Infrastructure/TmsApi.Infrastructure.c
```

```
sproj
dotnet sln TmsApi.sln add TmsApi.Api/TmsApi.Api.csproj
```

4. Wire the project references (dependencies point inward)

Enforce Clean Architecture boundaries by adding project-to-project references:

```
# Application references Domain (innermost)
dotnet add TmsApi.Application/TmsApi.Application.csproj reference TmsApi.Domain/TmsApi.Domain.csproj
```

```
# Infrastructure references Application and Domain
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj reference TmsApi.Application/TmsApi.Application.csproj
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj reference TmsApi.Domain/TmsApi.Domain.csproj
```

```
# Api references Application, Infrastructure, and Domain
dotnet add TmsApi.Api/TmsApi.Api.csproj reference TmsApi.Application/TmsApi.Application.csproj
dotnet add TmsApi.Api/TmsApi.Api.csproj reference TmsApi.Infrastructure/TmsApi.Infrastructure.csproj
dotnet add TmsApi.Api/TmsApi.Api.csproj reference TmsApi.Domain/TmsApi.Domain.csproj
```

Audit checkpoint: do this now. Run `dotnet build` from the root. The build will fail; that is expected: Steps 2–5 move the code. Confirm the error is only about unresolved using `TmsApi.Entities`; and using `TmsApi.Data`; namespaces, not about missing NuGet packages.

Step 2: Move the entities to `TmsApi.Domain`

The M6 monolith's `Entities/` folder is the cleanest move. Pure C# classes, no EF attributes, no NuGet. They belong in the innermost layer.

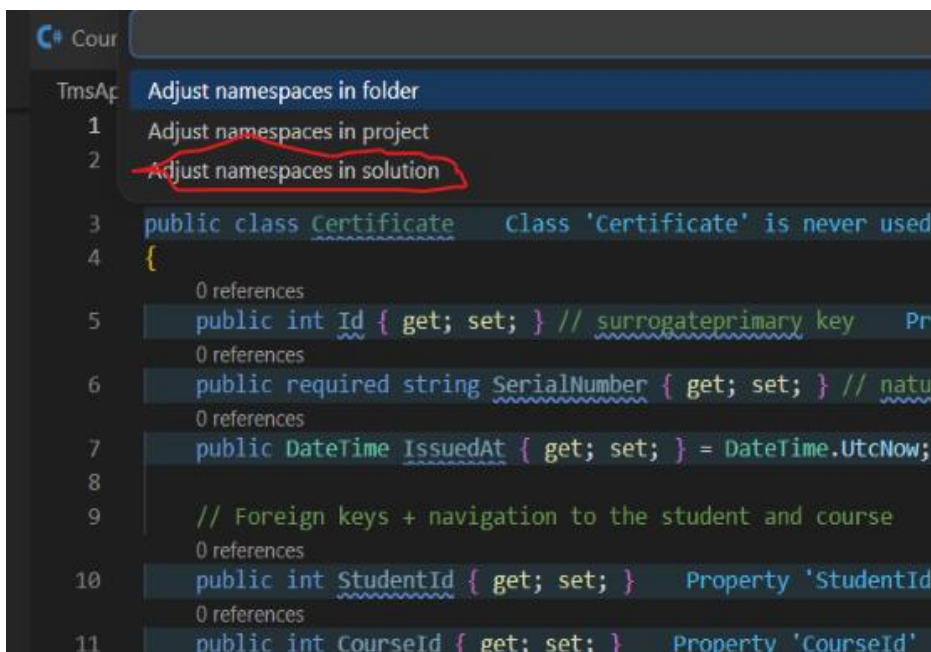
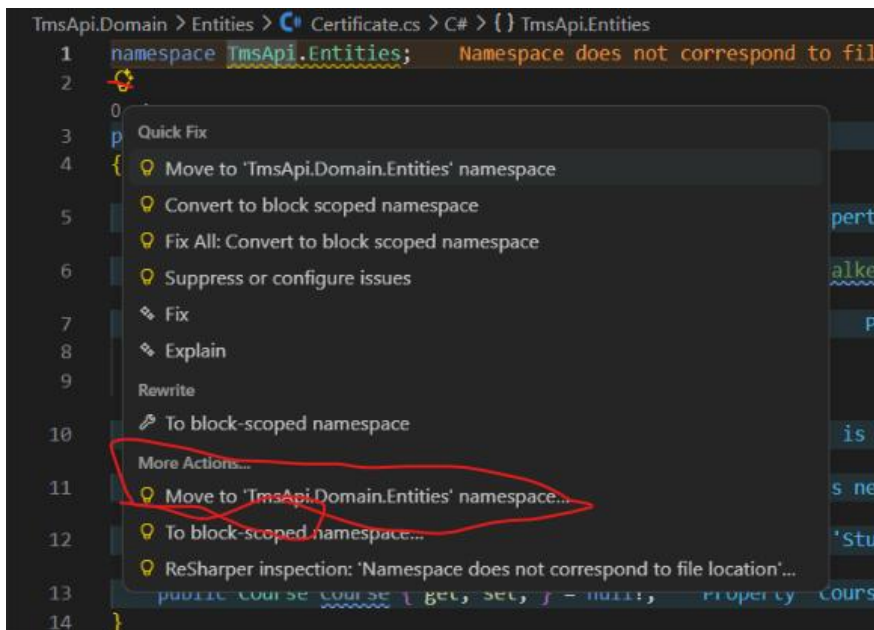
1. Create the entities folder and move the files

Run the following commands in Git Bash from the project root:

```
mkdir -p TmsApi.Domain/Entities
mv Entities/*.cs TmsApi.Domain/Entities/
```

2. Update the namespaces in the moved files

Open `TmsApi.Domain/Entities/Certificate.cs` in your editor. You will see the namespace declaration near the top of the file. Change the namespace declaration to the Clean Architecture format. Follow the editor as the image below



Audit checkpoint. dotnet build from the root should now fail with error CS5001: Program does not contain a static 'Main' method suitable for an entry point.

Step 3: Move TmsDbContext and EF configurations to TmsApi.Infrastructure

TmsDbContext and the IEntityTypeConfiguration<> classes are the EF-specific glue. They depend on Microsoft.EntityFrameworkCore and Npgsql.EntityFrameworkCore.PostgreSQL, both of which the Domain layer must not take on. The Infrastructure layer is their home.

1. Create the Infrastructure folder structure and move EF configurations

Run the following commands in Git Bash from the project root:

```
mkdir -p TmsApi.Infrastructure/Persistence/Configurations
mv Data/TmsDbContext.cs TmsApi.Infrastructure/Persistence/
mv Data/Configurations/* TmsApi.Infrastructure/Persistence/Configurations/ 2>/dev/null || true
mkdir -p TmsApi.Infrastructure/Services
mv Services/* TmsApi.Infrastructure/Services/
rmdir Data/Configurations 2>/dev/null || true
rmdir Data 2>/dev/null || true
```

2. Update namespaces in the moved files

Update the namespace declarations inside the moved files:

- Change namespace `TmsApi.Data;` to namespace `TmsApi.Infrastructure.Persistence;`
- Change using `TmsApi.Entities;` to using `TmsApi.Domain.Entities;` (Infrastructure is allowed to reference Domain)

3. Add Entity Framework Core packages to Infrastructure

Since the data model now lives in `TmsApi.Infrastructure`, you need to add the required EF Core NuGet packages to this project:

```
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj package Microsoft.EntityFrameworkCore
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj package Microsoft.EntityFrameworkCore.Design
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj package Microsoft.EntityFrameworkCore.Relational
dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj package Npgsql.EntityFrameworkCore.PostgreSQL
```

Now remove these packages from `Tms.Api.Csproj`

```
- <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.7" />
- <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />
```

4. Ensure Design package is on the Startup project

The `Microsoft.EntityFrameworkCore.Design` package is required on **both** `TmsApi.Infrastructure` (where the model lives) **and** `TmsApi.Api` (the startup project).

The `dotnet ef` CLI builds a design-time service provider by running the **startup project** (it does not walk project references to find Design at run time). Add the design package to `TmsApi.Api`:

```
dotnet add TmsApi.Api/TmsApi.Api.csproj package Microsoft.EntityFrameworkCore.Design
```

5. Update Program.cs DbContext registration

Update the using statement and context registration inside `TmsApi.Api/Program.cs`:

```
using TmsApi.Infrastructure.Persistence;

builder.Services.AddDbContext<TmsDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("TmsDatabase")));
```

6. Run migrations to verify database schema parity

Run these commands from the project root to verify your model is recognized properly:

```
dotnet ef migrations add PostSplitBaseline \
    --project TmsApi.Infrastructure/TmsApi.Infrastructure.csproj \
    --startup-project TmsApi.Api/TmsApi.Api.csproj
```

The generated migration file should be **empty** because the model shape did not change. Apply the migration to verify:

```
dotnet ef database update \
    --project TmsApi.Infrastructure/TmsApi.Infrastructure.csproj \
    --startup-project TmsApi.Api/TmsApi.Api.csproj
```

Audit checkpoint. The M6 database schema must be byte-identical after the move. Open pgAdmin (or psql) and confirm tables are present and unchanged.

Step 4: Move services, DTOs, and validators to TmsApi.Application

Services/ and DTOs/ from the M6 monolith are application-layer concerns.

1. Create the Application folder structure and move DTOs and Interfaces

Run the following commands in Git Bash from the project root:


```
mkdir -p TmsApi.Application/DTOs TmsApi.Application/Interfaces TmsApi.A  
pplication/Services  
mv DTOs/* TmsApi.Application/DTOs/  
mv Services/I*.cs TmsApi.Application/Interfaces/ 2>/dev/null || true
```

2. Determine where to place concrete service implementations

For concrete service classes, decide where they belong based on their dependencies:

- **Move to Application** (TmsApi.Application/Services/) if they only depend on interfaces and contain purely orchestration/business logic.
- **Move to Infrastructure** (TmsApi.Infrastructure/Persistence/) if they directly depend on TmsDbContext or external infrastructure/APIs.

3. Add FluentValidation package to Application

Since validators (which use AbstractValidator<> from FluentValidation) are moved to the Application project, you need to add the required package:

```
dotnet add TmsApi.Application/TmsApi.Application.csproj package FluentV  
alidation
```

[!IMPORTANT] **Namespace Import Warning:** Because FluentValidation is a third-party library, its namespace is not part of .NET's default implicit usings. Once the validators are moved to TmsApi.Application, you will likely get a compile error: error CS0246: The type or namespace name 'AbstractValidator<>' could not be found

To resolve this, open your validator files (e.g. CreateCourseValidator.cs) and ensure you add the using directive at the top:

```
using FluentValidation;
```

4. Update namespaces in the moved files like we did before

- TmsApi.DTOs → TmsApi.Application.DTOs
- TmsApi.Services (for interfaces) → TmsApi.Application.Interfaces
- TmsApi.Services (for implementations in Application) → TmsApi.Application.Services
- TmsApi.Services (for implementations in Infrastructure) → TmsApi.Infrastructure.Persistence
- TmsApi.Entities → TmsApi.Domain.Entities

Ensure that no old namespaces remain in your projects.

Option A: VS Code Search (OS-Agnostic / Editor-first) Press Ctrl + Shift + F (or Cmd + Shift + F on macOS), toggle **Regular Expression mode** (the .* icon in the search bar), and search for:

```
TmsApi\.(Services|DTOs|Data|Entities)
```

Option B: Terminal Search (Git Bash / macOS / Linux) Run grep from the project root:

```
grep -rn "TmsApi.Services\|TmsApi.DTOs\|TmsApi.Data\|TmsApi.Entities" TmsApi.Api/
```

Every hit must be rewritten to the new namespaces. None should remain.

Step 5: Move controllers and Program.cs into TmsApi.Api

The controllers, middleware, configurations, and startup settings belong in TmsApi.Api.

1. Create the Controllers folder and move files

Run the following commands in Git Bash from the project root:

```
mkdir -p TmsApi.Api/Controllers
mv Controllers/* TmsApi.Api/Controllers/
mkdir -p TmsApi.Api/Middlewares
mv Middlewares/* TmsApi.Api/Middlewares/
mkdir -p TmsApi.Api/Options
mv Options/* TmsApi.Api/Options/
mkdir -p TmsApi.Api/Properties
mv Properties/* TmsApi.Api/Properties/
mv Program.cs TmsApi.Api/ 2>/dev/null || true
mv appsettings.json TmsApi.Api/ 2>/dev/null || true
mv appsettings.Development.json TmsApi.Api/ 2>/dev/null || true
```

Note: Delete the empty directories

2. Update namespaces in controllers and Program.cs

Update controller namespaces:

- TmsApi.Controllers → TmsApi.Api.Controllers
- TmsApi.DTOs → TmsApi.Application.DTOs
- TmsApi.Services → TmsApi.Application.Interfaces (or TmsApi.Application.Services / TmsApi.Infrastructure.Persistence depending on where implementations landed)
- TmsApi.Entities → TmsApi.Domain.Entities

- TmsApi.Data → TmsApi.Infrastructure.Persistence

Update Program.cs to resolve dependency references using the correct Clean Architecture layers.

Audit checkpoint. dotnet build from the root should now succeed with **zero errors and zero warnings**.

Step 6: Run the M6 verification suite

Run these commands from the project root:

```
dotnet build
```

```
dotnet run --project TmsApi.Api/TmsApi.Api.csproj
```

Verify the course lists, course detail, and enrollment functionality using Scalar at the local API path.

Step 7: Commit the refactor and move to Session 1

Commit the clean architecture structure to Git:

```
git add -A
git commit -m "M7 Session 0: split M6 monolith into clean architecture
(4 projects)"
```

What you should now have

After this session, your workspace at the root should look like:

```
TmsApi.sln
TmsApi.Api/                ← web SDK, controllers, Program.cs, appset
tings
  Controllers/
  Program.cs
  appsettings.json
  appsettings.Development.json
TmsApi.Application/        ← classlib, DTOs + interfaces + MediatR ha
ndlers
  Common/
  DTOs/
  Interfaces/
  Services/
TmsApi.Infrastructure/     ← classlib, EF Core + repository implement
ations
  Persistence/
    TmsDbContext.cs
  Configurations/
```

TmsApi.Domain/
Entities/ ← classlib, pure entities, no NuGet

No file from the M6 monolith should be sitting at the root directory's Entities/, Data/, Services/, or DTOs/ anymore. If any old empty folder remains, remove it:

```
rmdir Entities Data Services DTOs Controllers 2>/dev/null || true
```

Session 0 Checkpoint

Tick all of these before you move to **Guided Lab Session 1**:

- ☐ TmsApi.sln exists at the root and references all four projects.
- ☐ TmsApi.Domain/Entities/ contains all five TMS entities with the TmsApi.Domain.Entities namespace.
- ☐ TmsApi.Application/ holds DTOs, service interfaces, and any in-Application service implementations.
- ☐ TmsApi.Infrastructure/Persistence/TmsDbContext.cs compiles and is reachable from Program.cs.
- ☐ Microsoft.EntityFrameworkCore.Design is referenced by **both** TmsApi.Infrastructure and TmsApi.Api (the startup project).
- ☐ TmsApi.Api/Controllers/ holds the M6 controllers with the TmsApi.Api.Controllers namespace.
- ☐ TmsApi.Api/Program.cs is the only Program.cs in the workspace.

If any box is unchecked, fix it before opening **Guided Lab Session 1**.

Why this is not a numbered exercise

M7 grades nine numbered exercises across four sessions (see **Module 7 Export Guide**, **Module 7 Workbook**, and the **LO-to-Evidence Mapping** in **Module 7 Assessment Pack**). Session 0 is intentionally not one of them:

- It is a **refactor of code the learner already wrote**, not new functionality. The grading rubric for the M7 capstone does not have a row for “split into clean architecture”; the rubric assumes the split has already happened.
- It is a **prerequisite**, framed in the same way the M6 labs use ## Before You Begin blocks: as the entry ticket to the next graded exercise.